```
from datasets import load_dataset

ds = load_dataset("Q-b1t/IMDB-Dataset-of-50K-Movie-Reviews-Backup")
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/se
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher r
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Plea
README.md: 100%  23.0/23.0 [00:00<00:00, 2.41kB/s]
archive.zip: 100%  27.0M/27.0M [00:02<00:00, 135MB/s]
Generating train split: 100%  50000/50000 [00:00<00:00, 52909.03 examples/s]
```

```
from datasets import load_dataset

# Load IMDB dataset
dataset = load_dataset("imdb")

# Check dataset structure
print(dataset)
```

```
README.md: 7.81k/? [00:00<00:00, 822kB/s]
plain_text/train-00000-of-00001.parquet: 100%  21.0M/21.0M [00:00<00:00, 105MB/s]
plain_text/test-00000-of-00001.parquet: 100%  20.5M/20.5M [00:00<00:00, 102MB/s]
plain_text/unsupervised-00000-of-00001.p(…): 100%  42.0M/42.0M [00:01<00:00, 68.5MB/s]
Generating train split: 100%  25000/25000 [00:00<00:00, 120015.11 examples/s]
Generating test split: 100%  25000/25000 [00:00<00:00, 120239.20 examples/s]
Generating unsupervised split: 100%  50000/50000 [00:00<00:00, 183137.41 examples/s]
DatasetDict({
    train: Dataset({
        features: ['text', 'label'],
        num_rows: 25000
    })
    test: Dataset({
        features: ['text', 'label'],
        num_rows: 25000
    })
    unsupervised: Dataset({
```

```
from transformers import DistilBertTokenizerFast

tokenizer = DistilBertTokenizerFast.from_pretrained("distilbert-base-uncased")

def tokenize(batch):
    return tokenizer(batch["text"], padding="max_length", truncation=True)

tokenized_dataset = dataset.map(tokenize, batched=True)
```

```
tokenizer_config.json: 100%  48.0/48.0 [00:00<00:00, 2.84kB/s]
vocab.txt: 232k/? [00:00<00:00, 1.05MB/s]
tokenizer.json: 466k/? [00:00<00:00, 1.26MB/s]
Map: 100%  25000/25000 [00:17<00:00, 1319.89 examples/s]
Map: 100%  25000/25000 [00:17<00:00, 1542.96 examples/s]
Map: 100%  50000/50000 [00:36<00:00, 1484.83 examples/s]
```

```
train_dataset = tokenized_dataset["train"]
test_dataset = tokenized_dataset["test"]
```

```
import torch

class SentimentDataset(torch.utils.data.Dataset):
    def __init__(self, encodings, labels):
        self.encodings = encodings
        self.labels = labels

    def __len__(self):
        return len(self.labels)

    def __getitem__(self, idx):
        item = {key: torch.tensor(val[idx]) for key, val in self.encodings.items()}
        item["labels"] = torch.tensor(self.labels[idx])
        return item

# Extract encodings and labels from the tokenized_dataset
train_encodings = {
    'input_ids': tokenized_dataset["train"]['input_ids'],
    'attention_mask': tokenized_dataset["train"]['attention_mask']
}
train_labels = tokenized_dataset["train"]['label']

test_encodings = {
    'input_ids': tokenized_dataset["test"]['input_ids'],
    'attention_mask': tokenized_dataset["test"]['attention_mask']
}
test_labels = tokenized_dataset["test"]['label']

train_dataset = SentimentDataset(train_encodings, train_labels)
test_dataset = SentimentDataset(test_encodings, test_labels)
```

```
from transformers import pipeline

# Load pretrained sentiment analysis pipeline (fast, no training)
sentiment_pipeline = pipeline("sentiment-analysis")

# Test on a few sample sentences
texts = [
    "I loved this movie, it was fantastic!",
    "The film was boring and too long.",
    "An average experience, not too bad but not great."
]

results = sentiment_pipeline(texts)
for text, res in zip(texts, results):
    print(f"Text: {text}\nPrediction: {res}\n")
```

```
No model was supplied, defaulted to distilbert/distilbert-base-uncased-finetuned-sst-2-english and revisi
Using a pipeline without specifying a model name and revision in production is not recommended.
```

config.json: 100% 629/629 [00:00<00:00, 68.9kB/s]

model.safetensors: 100% 268M/268M [00:05<00:00, 54.7MB/s]

Loading weights: 100% 104/104 [00:00<00:00, 440.21it/s, Materializing param=pre_classifier.weight]

tokenizer_config.json: 100% 48.0/48.0 [00:00<00:00, 4.78kB/s]

vocab.txt: 232k/? [00:00<00:00, 12.2MB/s]

```
Text: I loved this movie, it was fantastic!
Prediction: {'label': 'POSITIVE', 'score': 0.9998804330825806}

Text: The film was boring and too long.
Prediction: {'label': 'NEGATIVE', 'score': 0.9997709393501282}

Text: An average experience, not too bad but not great.
Prediction: {'label': 'NEGATIVE', 'score': 0.9976065158843994}
```

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score

# Tiny subset for speed
train_texts = dataset["train"]["text"][:1000]
train_labels = dataset["train"]["label"][:1000]
test_texts = dataset["test"]["text"][:500]
test_labels = dataset["test"]["label"][:500]

vectorizer = CountVectorizer(stop_words="english", max_features=2000)
X_train = vectorizer.fit_transform(train_texts)
X_test = vectorizer.transform(test_texts)

nb = MultinomialNB()
nb.fit(X_train, train_labels)
preds = nb.predict(X_test)

print("Naive Bayes Accuracy:", accuracy_score(test_labels, preds))
```

```
Naive Bayes Accuracy: 1.0
```

```
import torch.nn as nn

class CNNModel(nn.Module):
    def __init__(self, vocab_size, embed_dim, num_classes):
        super(CNNModel, self).__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim)
        self.conv1 = nn.Conv1d(embed_dim, 100, kernel_size=3)
        self.pool = nn.MaxPool1d(2)
        self.fc = nn.Linear(100, num_classes)

    def forward(self, x):
        x = self.embedding(x)
        x = x.permute(0, 2, 1)
        x = self.conv1(x)
        x = self.pool(x)
        x = torch.relu(x)
        x = x.mean(dim=2)
        return self.fc(x)

model_cnn = CNNModel(vocab_size=2000, embed_dim=50, num_classes=2)
print(model_cnn)
```

```
CNNModel(
  (embedding): Embedding(2000, 50)
  (conv1): Conv1d(50, 100, kernel_size=(3,), stride=(1,))
  (pool): MaxPool1d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (fc): Linear(in_features=100, out_features=2, bias=True)
)
```